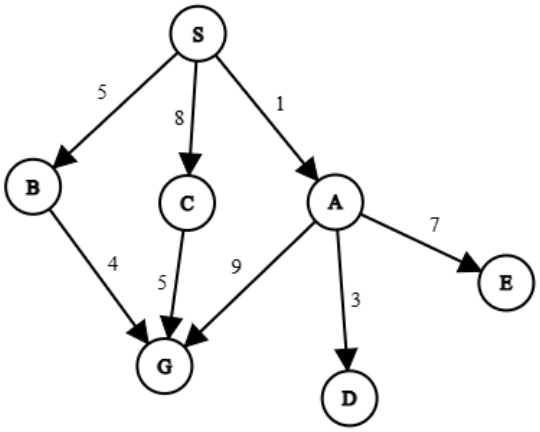

A directed Graph where node S is start, node G is goal node.

Heuristic
S 8
A 8
B 4
C 3
D 1000000000
E 1000000000
G 0

Experiment-01: Best First Search

- Best-First Search is an "informed" graph traversal algorithm that selects the next node to explore based on an evaluation function, rather than a fixed order like breadth or depth.
- It uses a heuristic to estimate which node is most likely to lead to the goal quickly, making it significantly faster than "blind" searches for complex problems.

Source Code:

```
import heapq
graph={}
parent={}
distance={}
heuristic={}

with open("graph.txt","r") as file: # storing the graph in nested dictionary
    f=file.readlines()
    for i in f:
        i=i.split(' ')
        if i[0] not in graph:
            graph[i[0]]={}
            distance[i[0]]=1e10
        graph[i[0]][i[1]]=int(i[2])
        if i[1] not in graph:
            graph[i[1]]={}
            distance[i[1]]=1e10
file.close()
```

```
# print(graph)
with open("heuristic.txt","r") as file: # storing the graph in nested dictionary
    f=file.readlines()
    for i in f:
        i=i.split(' ')
        heuristic[i[0]]=int(i[1])
file.close()
# print(heuristic)

start='S'#input("Start node: ")
goal='G'#input("Goal node: ")
path=""
cost=0

distance[start]=0
queue=[]
heapq.heappush(queue,(heuristic[start],start))

while len(queue)!=0:
    top=heapq.heappop(queue)
    if top[1]==goal:
        break
    for i in graph[top[1]]:
        distance[i]=graph[top[1]][i]+distance[top[1]]
        parent[i]=top[1]
        heapq.heappush(queue,(heuristic[i],i))

while True:
    path+=goal
    next_path=parent[goal]
    cost+=graph[next_path][goal]
    goal=next_path
    if goal==start:
        path+=goal
        break

print(f'path:{path[::-1]}\ncost={cost}')
```

Output:

```
path:SCG
cost=13
PS F:\RUET CSE LAB\3208-Artificial Intelligence\3. 22-10-2025>
```

Conclusion:

In conclusion, Best-First Search is a heuristic-driven strategy that prioritizes efficiency and speed by exploring the most promising paths first. By utilizing a priority queue and an estimation function, it drastically reduces the search space compared to uninformed algorithms, making it ideal for scenarios where finding *a* solution quickly is more important than finding the absolute *best* solution. Key Applications are Pathfinding in Video Games, Web Crawling (Focused Crawling), AI Planning and Scheduling, Speech Recognition, Route Finding in GPS Navigation Systems.

Experiment-02: Uniform Cost Search (UCS)

- Uniform Cost Search (UCS) is an uninformed search algorithm used for traversing weighted graphs.
- While Breadth-First Search (BFS) finds the path with the fewest steps, UCS finds the path with the lowest total cost.
- It expands nodes in concentric contours of increasing cost from the start node, ensuring that the first time it reaches the destination, it has found the cheapest possible path.

Source Code:

```
import heapq
graph={}
parent={}
distance={}

with open("graph.txt","r") as file: # storing the graph in nested dictionary
   f=file.readlines()
   for i in f:
      i=i.split(' ')
      if i[0] not in graph:
         graph[i[0]]={}
         distance[i[0]]=1e10
      graph[i[0]][i[1]]=int(i[2]) # edge is directed
      if i[1] not in graph:
         graph[i[1]]={}         # store the child also as key
         distance[i[1]]=1e10
file.close()
# print(graph)
start='S'#input("Start node: ")
goal='G'#input("Goal node: ")
path=""
cost=0

distance[start]=0
queue=[]
heapq.heappush(queue,(0,start))
```

```
while len(queue)!=0:
    top=heapq.heappop(queue)
    if top[1]==goal:
        break
    for i in graph[top[1]]:
        if distance[i]>(graph[top[1]][i]+distance[top[1]]): # if the distance is less than before then change the distance
            distance[i]=graph[top[1]][i]+distance[top[1]]
            parent[i]=top[1]
            heapq.heappush(queue,(distance[i],i))

while True:
    path+=goal
    next_path=parent[goal]
    cost+=graph[next_path][goal]
    goal=next_path
    if goal==start:
        path+=goal
        break
print(f'path:{path[::-1]}\ncost={cost}')
```

Output:

```
path:SBG
cost=9
PS F:\RUET CSE LAB\3208-Artificial Intelligence\3. 22-10-2025>
```

Conclusion:

In conclusion, Uniform Cost Search (UCS) is the definitive algorithm for finding the optimal path in weighted graphs where edge costs vary. By systematically exploring the "cheapest" nodes first using a cumulative cost function, it guarantees the most efficient solution, effectively acting as a generalized version of Breadth-First Search for complex environments. Key Applications are Route Planning in Navigation Systems, Network Routing Protocols, Solving Weighted Puzzles, Pipeline and Utility Layout Optimization, Robotics Path Planning in uneven terrain.

Experiment-03: A* Search

- A* Search is widely considered the most popular and powerful pathfinding algorithm in computer science.
- It is an informed search algorithm that combines the strengths of UCS and Greedy Best-First Search.
- A* finds the shortest path by evaluating nodes based on a combination of the cost already incurred and the estimated cost remaining.
- **The Core Formula: f(n) = g(n) + h(n)**

Source Code:

```
import heapq
graph={}
parent={}
distance={}
heuristic={}

with open("graph.txt","r") as file: # storing the graph in nested dictionary
    f=file.readlines()
    for i in f:
        i=i.split(' ')
        if i[0] not in graph:
            graph[i[0]]={}
            distance[i[0]]=1e10
        graph[i[0]][i[1]]=int(i[2])
        if i[1] not in graph:
            graph[i[1]]={}
            distance[i[1]]=1e10
file.close()
# print(graph)
with open("heuristic.txt","r") as file: # storing the graph in nested dictionary
    f=file.readlines()
    for i in f:
        i=i.split(' ')
        heuristic[i[0]]=int(i[1])
file.close()
# print(heuristic)

start='S'#input("Start node: ")
goal='G'#input("Goal node: ")
path=""
cost=0

distance[start]=0
queue=[]
heapq.heappush(queue,(distance[start]+heuristic[start],start))

while len(queue)!=0:
    top=heapq.heappop(queue)
    if top[1]==goal:
        break
    for i in graph[top[1]]:
        distance[i]=graph[top[1]][i]+distance[top[1]]
        parent[i]=top[1]
        heapq.heappush(queue,(distance[i]+heuristic[i],i))
```

```
while True:
    path+=goal
    next_path=parent[goal]
    cost+=graph[next_path][goal]
    goal=next_path
    if goal==start:
        path+=goal
        break
print(f'path:{path[::-1]}\ncost={cost}')
```

Output:

```
path:SBG
cost=9
PS F:\RUET CSE LAB\3208-Artificial Intelligence\3. 22-10-2025>
```

Conclusion:

In conclusion, A* Search is the industry-standard algorithm for pathfinding because it intelligently balances the actual cost of travel with an estimated cost to the target. By utilizing the function f(n) = g(n) + h(n), it achieves optimal results with maximum efficiency, avoiding the pitfalls of blind searches while maintaining the speed of heuristic approaches. Key Applications are Video Game Pathfinding, GPS Navigation and Traffic Routing systems, Robotics and Autonomous Vehicle Navigation, Natural Language Processing (Parsing), Solving Complex Puzzles.